

9 – Automation & Updates (cont.)

Task	Detail
Daily Image Refresh	Systemd timer <code>pod-update.timer</code> executes <code>/usr/local/bin/pod_update_all.sh</code> . The script pulls the latest image for each service, stops the old container, removes it, and restarts with the saved run command from <code>/etc/pod_run_commands/<service></code> . This keeps all containers up to date without downtime.
Watchtower (Optional)	Run <code>podman run -d --name watchtower \</code>

```
-v /var/run/docker.sock:/var/run/docker.sock \
containerrr/watchtower --interval 3600` for real-time monitoring of image updates. It will automatically
pull new tags and restart containers when a newer version appears.
```

10 – Testing & Validation

Test	Purpose	Method
Unit Tests	Verify individual service logic (e.g., Corda RPC, LM Studio inference).	Use <code>pytest</code> for Python services; Java JUnit for Corda. Run in a temporary container with test data.
Integration Tests	End-to-end flow: Drone → MeshChatX → OpenClaw → LM Studio → Mastodon.	Simulate LoRa packets using a PC-based LoRa emulator, send image files via SCP, and assert that a new post appears on the Mastodon instance with expected content.
Performance Benchmarks	Ensure 24/7 uptime under load (e.g., 10 concurrent drone uploads).	Use <code>ab</code> or <code>wrk</code> against WebODM API; monitor CPU/memory in Podman stats.
Failover Tests	Simulate pod crash and verify auto-restart.	Kill a container (<code>podman kill <name></code>), check that systemd restarts it within seconds.
Security Audits	Validate TLS, firewall rules, and least-privilege mounts.	Run <code>ss -tulpn</code> to confirm only expected ports open; use <code>goss</code> or <code>OpenSCAP</code> for container hardening checks.

11 – Documentation & Hand-Off

- README.md** – high-level architecture, prerequisites, and build steps.
- Deployment Guide** – step-by-step instructions to set up the host, create users, install dependencies, and deploy each pod (include sample `pod_run_commands` files).
- Configuration Reference** – list of all environment variables per service with default values and explanations.
- Operational Runbook** – troubleshooting common issues: service not starting, container OOM, database connection errors.
- Backup & Recovery** – scripts to snapshot PostgreSQL data, export simulation results, and restore from a backup.
- Change Log** – track versioning of images, configuration changes, and deployment dates.

12 – Risk Assessment & Mitigation

Risk	Impact	Likelihood	Mitigation
Container OOM (e.g., LM Studio GPU memory)	Service crash	Medium	Set <code>--memory</code> limits; monitor with <code>podman stats</code> ; auto-restart via systemd.
LoRa packet loss	Incomplete telemetry	Low	Implement redundancy: store packets locally on Pi, resend if not acknowledged.
Unauthorized access to Mastodon API	Data breach	Low	Store OAuth tokens in encrypted <code>.config/mastodon</code> , use HTTPS only, restrict IP ranges.
Disk space exhaustion (image ingestion)	System halt	Medium	Auto-rotate logs, set quota on <code>/home/scottw/drone_images</code> via <code>quota</code> or <code>edquota</code> .

Risk	Impact	Likelihood	Mitigation
Dependency breakage (e.g., image pull failure)	Deployment freeze	Low	Pin image tags; maintain local registry mirror; include fallback script to retry pulls.

13 – Timeline & Milestones

Week	Deliverable
0	Environment set up, Podman installed.
1	Core network and user accounts created.
2	Systemd unit templates finalized; first pod (Corda) deployed.
3	MeshChatX + LoRa integration complete.
4	WebODM image ingestion pipeline operational.
5	LM Studio & OpenClaw AI stack functional.
6	Gazebo/ArduPilot simulation running and exporting results.
7	Automation scripts (<code>pod_update_all.sh</code> , timers) in place.
8	Full end-to-end testing, performance benchmarks, security audit.
9	Documentation hand-off, operational runbook finalized.

14 – Acceptance Criteria

- All services start automatically on boot and remain healthy for 72 hours without manual intervention.
- A drone can transmit a LoRa packet, upload an image via Wi-Fi, have it processed by WebODM, trigger a Mastodon post through OpenClaw, and receive payment confirmation in Corda within 5 minutes.
- The simulation pod runs at least one full mission cycle, outputs JSON logs to `/results` , and those files are accessible from the host filesystem.
- Systemd timers successfully pull newer images and restart pods without downtime during a simulated update cycle.
- Security audit passes with no exposed ports beyond the defined public list.

15 – Next Steps for the Agentic AI

1. **Implement Scripts** – Create `/usr/local/bin/pod_update_all.sh` that iterates over `/etc/pod_run_commands/*` . Ensure idempotent restart logic.
2. **Write Systemd Units** – For each service, generate a unit file that references the run command and sets `Restart=on-failure` .
3. **Configure Traefik** – Deploy as a pod with dynamic config files mapping public ports to internal pods. Test HTTPS with Let's Encrypt.
4. **Deploy LoRa Stack** – Install `lora-tools` , configure the radio hat, and validate packet reception in MeshChatX logs.
5. **Set Up WebODM** – Pull `webodm:latest` , mount `/home/scottw/webodm_output` , and verify that new images trigger processing jobs automatically.
6. **Deploy AI Stack** – Build LM Studio image (`python:3.11-slim` + PyTorch), expose 5000, run OpenClaw container with access to Mastodon credentials.
7. **Simulation Setup** – Install ROS2 Foxy, Gazebo 11, SB3; create a world file, launch ArduPilot container, and confirm telemetry over MAVLink UDP.
8. **Automation & Monitoring** – Configure `pod-update.timer` , enable Watchtower if desired, set up Traefik health checks.
9. **Testing** – Run unit, integration, performance, failover tests; document results.
10. **Documentation** – Populate README, deployment guide, configuration reference, runbook, backup scripts.